



Урок 7 > Масштабирование системы

> Масштабирование системы

Ранее мы масштабировали систему в виде увеличения количества составляющих. Мы также определялись с типами хранилищ под различные задачи, добавляли очереди сообщений и внешние сервисы.

Но что делать, если система **не выдерживает** рассчитанные на предварительном этапе нагрузки?

Самое простое решение — продублировать систему, но это будет работать только для самых простейших приложений. Для более комплексных приложений мы не можем просто разбить систему на большие повторяющиеся части, потому что между ними должна быть синхронизация (например, если соцсеть разбить на 4 части, то по сути пользователи будут заходить в 4 разных соцсети).

В одной из первых лекций мы разбирали приблизительные ограничения по нагрузке:

- Сеть: С10k давно не вопрос, 1Gbps даже в облаке, платим не более \$0.1/GB
- Вычисления: отдаём 100k текста, считываем 10k и записываем 1k простых запросов в БД
- Хранилище: HDD 300 MB/s и 30/TB, SSD 5 GB/s и 300/TB, RAM 50GB/s и \$10000/TB

Допустим, мы поняли, что нагрузка возросла и в рамках какого-то пункта нам надо много машин, что будем делать?

В этом уроке мы разберём, как при увеличении количества машин:

-
- распределять нагрузку между сервисами;
 - распределять данные внутри хранилища;

- переживать выход части машин из строя;
- избегать при этом перекосов в нагрузке.

> Балансировка нагрузки

Балансировка нагрузки — это метод распределения заданий между несколькими сетевыми устройствами (например, серверами) с целью оптимизации использования ресурсов, сокращения времени обслуживания запросов, горизонтального масштабирования кластера (динамическое добавление/удаление устройств), а также обеспечения отказоустойчивости (резервирования).

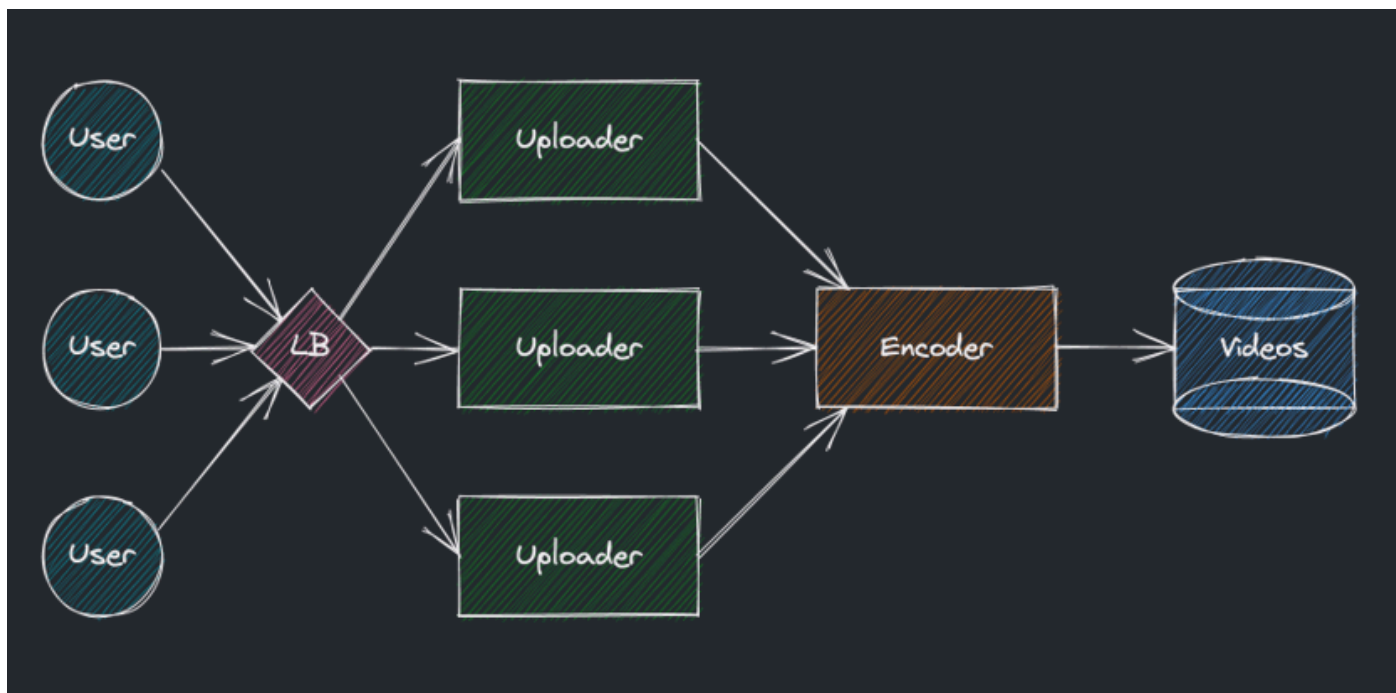
В компьютерах балансировка распределяет нагрузку между несколькими вычислительными ресурсами, такими как компьютеры, компьютерные кластеры, сети, центральные процессоры или диски. Цель балансировки нагрузки — оптимизация использования ресурсов, максимизация пропускной способности, уменьшение времени отклика и предотвращение перегрузки какого-либо одного ресурса.

Использование нескольких компонентов балансировки нагрузки вместо одного компонента может повысить надёжность и доступность за счёт резервирования. Балансировка нагрузки предполагает обычно наличие специального программного обеспечения или аппаратных средств, таких как многоуровневый коммутатор или система доменных имён, как серверный процесс.

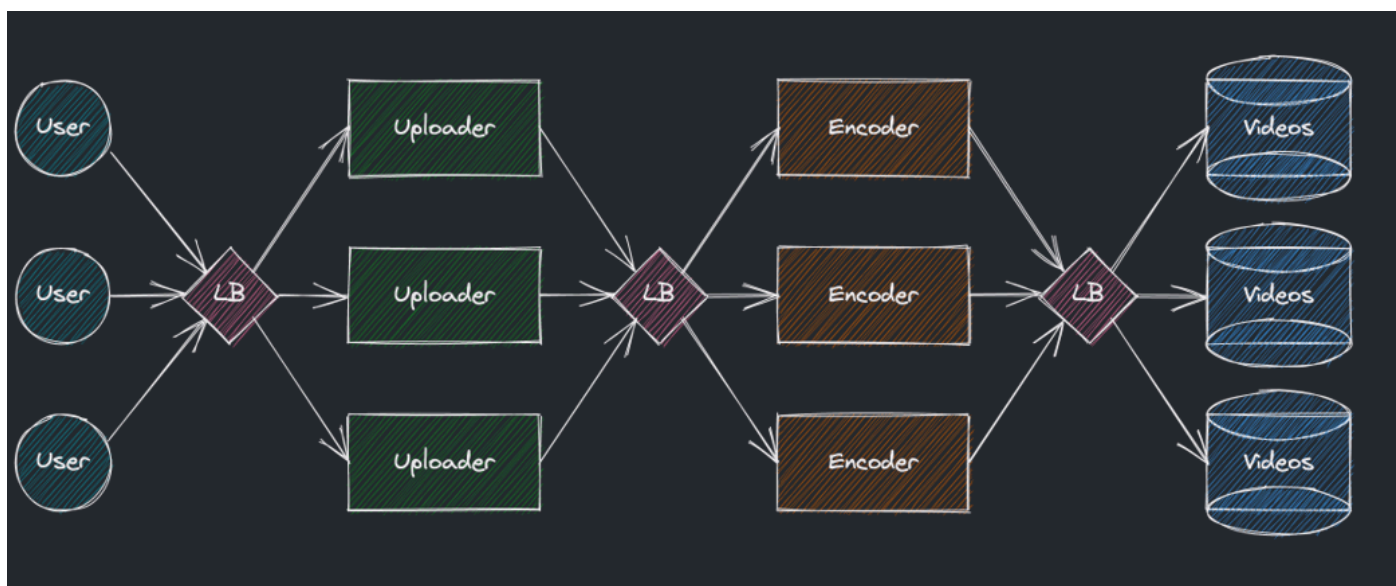
Балансировщик нагрузки (load balancer) — критически важный компонент любой распределённой системы, который распределяет передаваемый трафик между разными приложениями, сервисами и хранилищами.

Балансировщик также может следить за статусом сервисов, в которые он передаёт запросы, чтобы перестать передавать запросы к серверам, переставшим отвечать в течение заданного времени.

Естественной задачей для балансировщика является балансировка пользовательского трафика — распределение входящих запросов от пользователя между инстансами сервиса:



Но для максимальной надёжности они могут располагаться между каждой парой уровней логики системы:



Принцип работы балансировщика нагрузки

- Анализ того, какие из серверов готовы принять и обработать запрос пользователя;
- Выбор наиболее подходящего сервера по заданному алгоритму.

Алгоритм выбора подходящего сервера

- По количеству подключений — приоритет серверу с минимальным количеством подключений;
- По времени ответа — приоритет наименьшей задержке перед ответом;

- По трафику — приоритет серверу с минимальной сетевой нагрузкой;
 - Round-robin — на каждый запрос меняем сервер по кругу из заранее определённой очереди;
 - Weighted round-robin — серверы получают веса на основе комбинации из характеристик выше;
 - Пользовательский хэш — выбираем сервер на основе хэша (IP, user_id, ...).
-

Преимущества использования балансировщика

- Улучшается пользовательский опыт за счёт передачи запроса подходящему серверу;
- Снижается нагрузка на каждый из серверов за счёт распределения запросов между ними;
- Справляемся с отказом части серверов, по крайней мере, в случае stateless сервиса;
- Умные балансировщики могут даже предсказывать рост нагрузки и инициировать автоскейлинг.

> Распределение данных

При разрастании популярности сервиса растёт и объём данных, которые необходимо хранить, к которым обеспечиваем быстрый и надёжный доступ множеству пользователей. Для распределения данных есть несколько основных подходов:

- Секционирование (partitioning, партиционирование) — разделение больших хранимых файлов или объектов в базах данных, таких как таблицы, индексы и пр. на логические части по каким-то критериям. Секционирование повышает масштабируемость и производительность системы;
- Избыточность и репликация. Под избыточностью понимается дублирование важных элементов системы для повышения надёжности. Под репликацией понимается копирование данных в избыточные сервисы для консистентности.

> Секционирование

Выделяют несколько видов секционирования:

- Вертикальное секционирование (по умолчанию) подразумевает выделение разных смысловых частей данных и последующее вероятное разнесение их по разным машинам, запас тем самым ограничен;
 - Горизонтальное секционирование, оно же шардирование (sharding) подразумевает разбиение данных на уровне таблиц по выбранному ключу, например, дню недели или почтовому индексу.
-

Способы секционирования записей

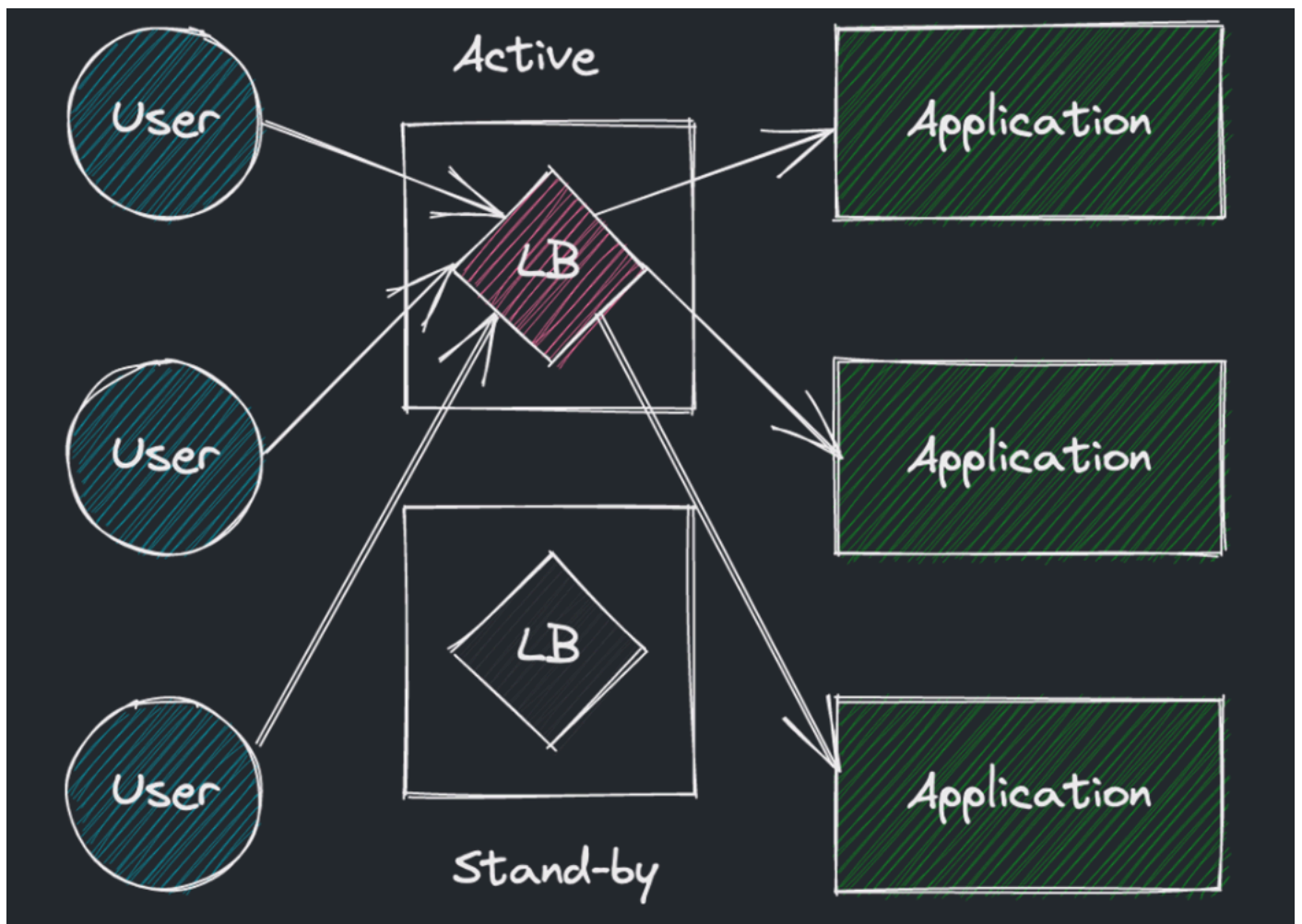
- На основе ключа или хеша: применяем хеш-функцию к какому-либо из полей и получаем номер партии. Например, если мы располагаем сотней серверов и сохраняем время создания записей как `int`, то номер партии можно определить по формуле «`timestamp % 100`». При удачном выборе такого ключа (не `weekday % 7`) данные будут распределены предопределённым образом и равномерно. Из минусов — фиксированное количество серверов. Выпадет один и начнется перекос в нагрузке;
 - По спискам — мы можем заранее распределить все поступающие записи по предопределённым группам на основе страны, дня недели и т.п., понимая, как соотносится нагрузка из разных групп;
 - Round-robin разбиение — меняем партии по кругу с каждой новой поступающей записью;
 - Составное разбиение — комбинируем что-либо из вышеупомянутого.
-

Недостатки метода

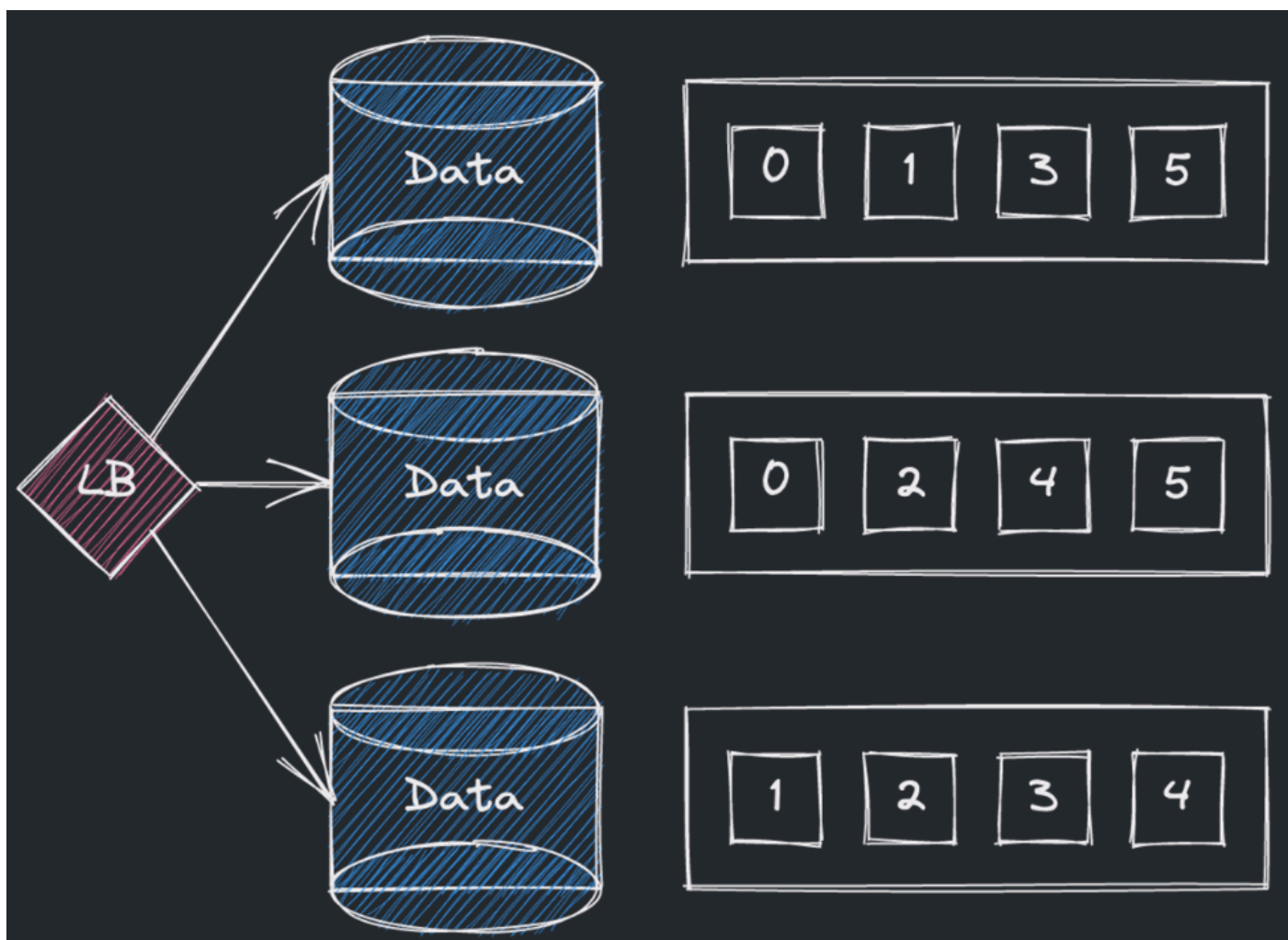
- Выполнение JOIN-ов — они теперь затрагивают данные на нескольких серверах и часто невыполнимы. Частично решается денормализацией БД;
- Большинство РСУБД не могут следить за валидностью внешних ключей на разных серверах;
- Ребалансировка — иногда нам нужно изменить способ шардирования, если поздно поняли уязвимость текущего способа, и как следствие получили неравномерность данных или запросов по серверам. Скорее всего это приведёт к пересылке всех данных между серверами и недоступности все это время.

> Избыточность и репликация

Избыточность



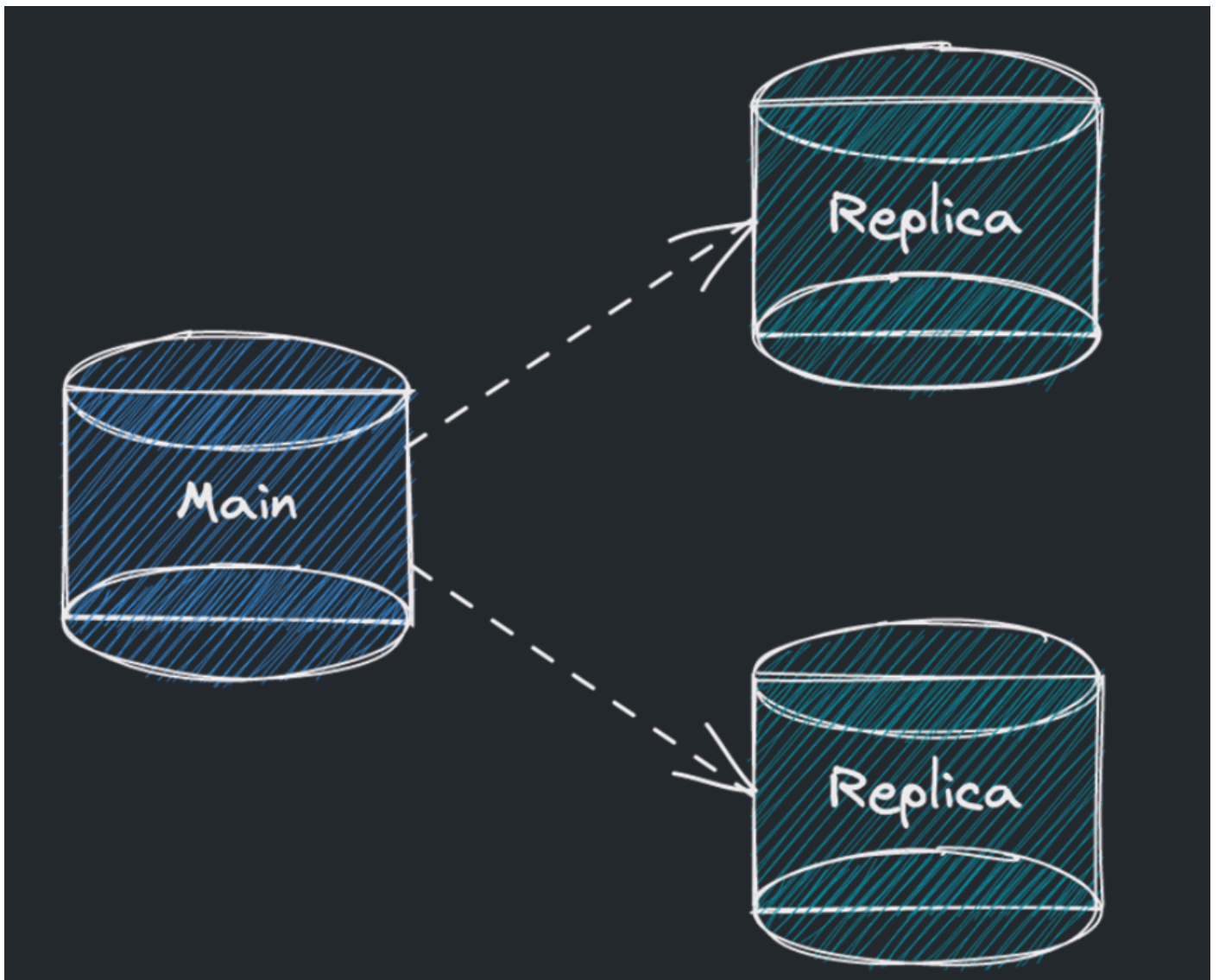
Резервный сервис в пару к действующему на отдельной машине



Размноженные копии файлов с заданным коэффициентом репликации.

Репликация

Репликация повышает надёжность, устойчивость к отказам и доступность системы.



При репликации баз данных у нас есть множество разных копий одних и тех же данных.

Но как гарантировать их консистентность для всех пользователей? Есть две стратегии:

- Кворум
- Лидер и последователи

Кворум

Некий состоящий из $\lfloor N/2 \rfloor + 1$ узлов уровень согласованности носит название кворума, большинства узлов. В случае разбиения сетевой среды на части или отказа узлов, в системе с $2f + 1$ узлами, живые узлы способны продолжать приём чтений и записей при недоступности до f узлов до тех пор, пока оставшаяся часть кластера не станет снова доступной. Иными словами, такая система безразлична к отказу не более f узлов.

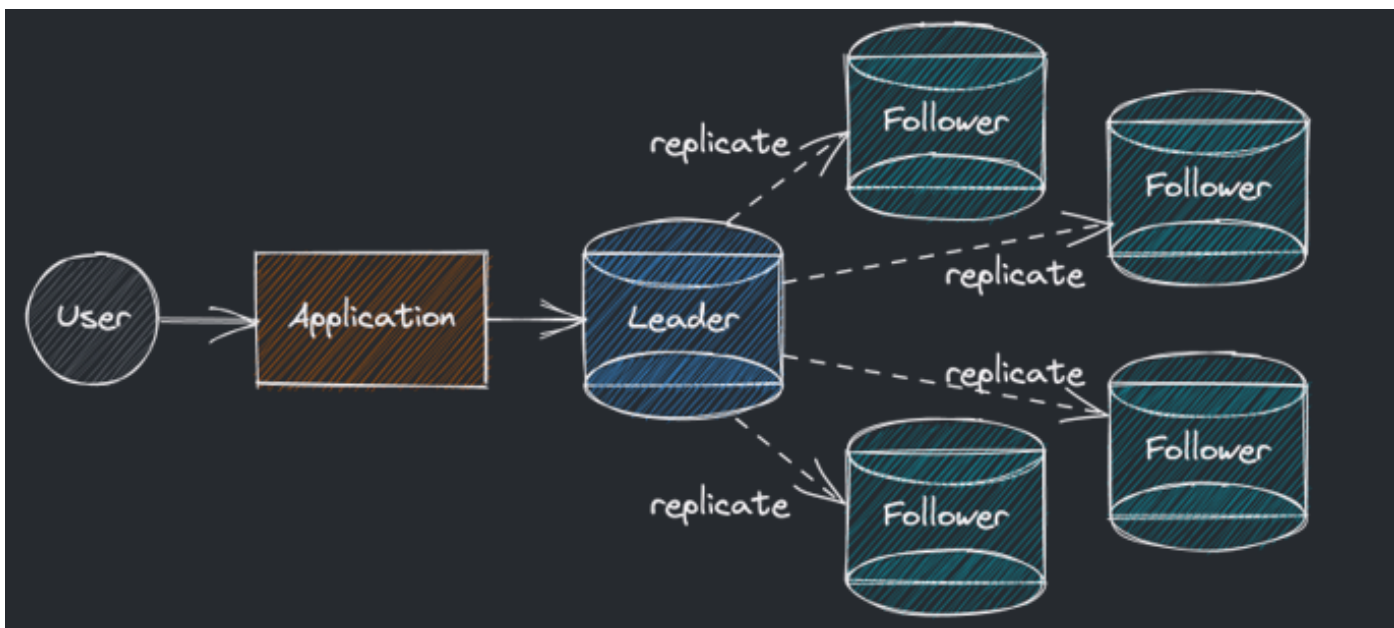
Выполняя операцию чтения и записи с применением кворума, система не может быть безразлична к отказу большинства узлов. Например, когда в сумме имеются три реплики, а две из них упали, операции чтения и записи не способны достичь значения числа узлов, требуемого для

согласованности чтения и записи, ибо лишь один узел из трёх способен отвечать на соответствующие запросы.

Достижение кворума как критерий успешности операции гарантирует консистентность данных. Тем не менее необходимость достижения кворума влечет и эпизодическую недоступность сервиса.

Лидер и последователи

В каждый момент времени есть только один лидер, который осуществляет репликацию данных. Последователи пишут данные по запросу лидера, служат резервной копией и помогают при чтении. В случае отказа лидера один из последователей становится новым лидером:

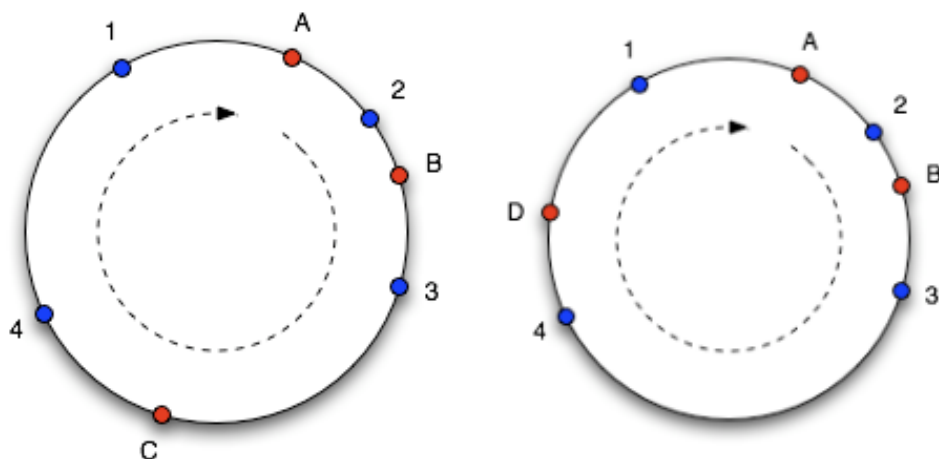


Лидер и последователи

> Консистентное (согласованное) хеширование

При использовании стандартных методов распределения данных между серверами может возникнуть ситуация, когда один или несколько серверов становятся недоступны. Тогда приходится перестраивать логику распределения данных. Такой подход очень ресурсоёмкий и не позволяет добиться хороших показателей по скорости ответа системы. С данной проблемой может помочь подход, называемый согласованным или консистентным хешированием.

Согласованное хеширование (англ. consistent hashing) — особый вид хеширования, отличающийся тем, что когда хеш-таблица перестраивается, только K/n ключей в среднем должны быть переназначены, где K — число ключей и n число слотов (slots, buckets). В противоположность этому, в большинстве традиционных хеш-таблиц, изменение количества слотов вызывает переназначение почти всех ключей.



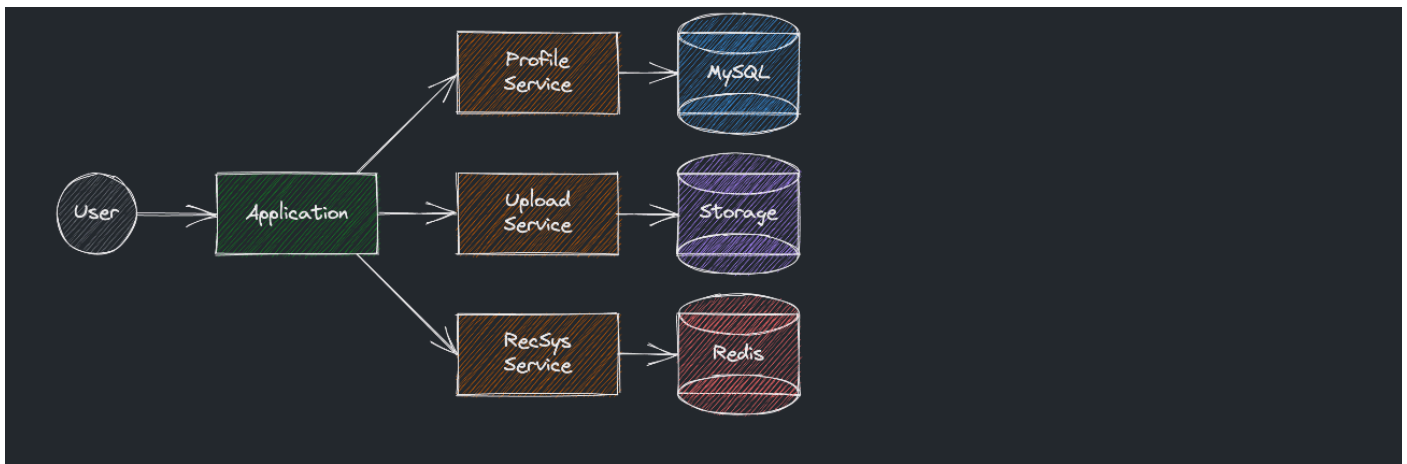
Суть алгоритма заключается в следующем: мы рассматриваем набор целых чисел от 0 до 2^{32} , «закручивая» числовую ось в кольцо (склеиваем 0 и 2^{32}). Каждому серверу из пула серверов мы сопоставляем число на кольце (рисунок слева, сервера A, B и C). Ключ хешируется в число в том же диапазоне (на рисунке – синие точки 1-4), в качестве сервера для хранения ключа мы выбираем сервер в точке, ближайшей к точке ключа в направлении по часовой стрелке. Если сервер удаляется из пула или добавляется в пул, на оси появляется или исчезает точка сервера, в результате чего лишь часть ключей перемещается на другой сервер.

На рисунке 2 справа показана ситуация, когда сервер C был удалён из пула серверов и добавлен новый сервер D. Легко заметить, что ключи 1 и 2 не поменяли привязки к серверам, а ключи 3 и 4 переместились на другие сервера. На самом деле одному серверу ставится в соответствие 100-200 точек на оси (пропорционально его весу в пуле), что улучшает равномерность распределения ключей по серверам в случае изменения их конфигурации.

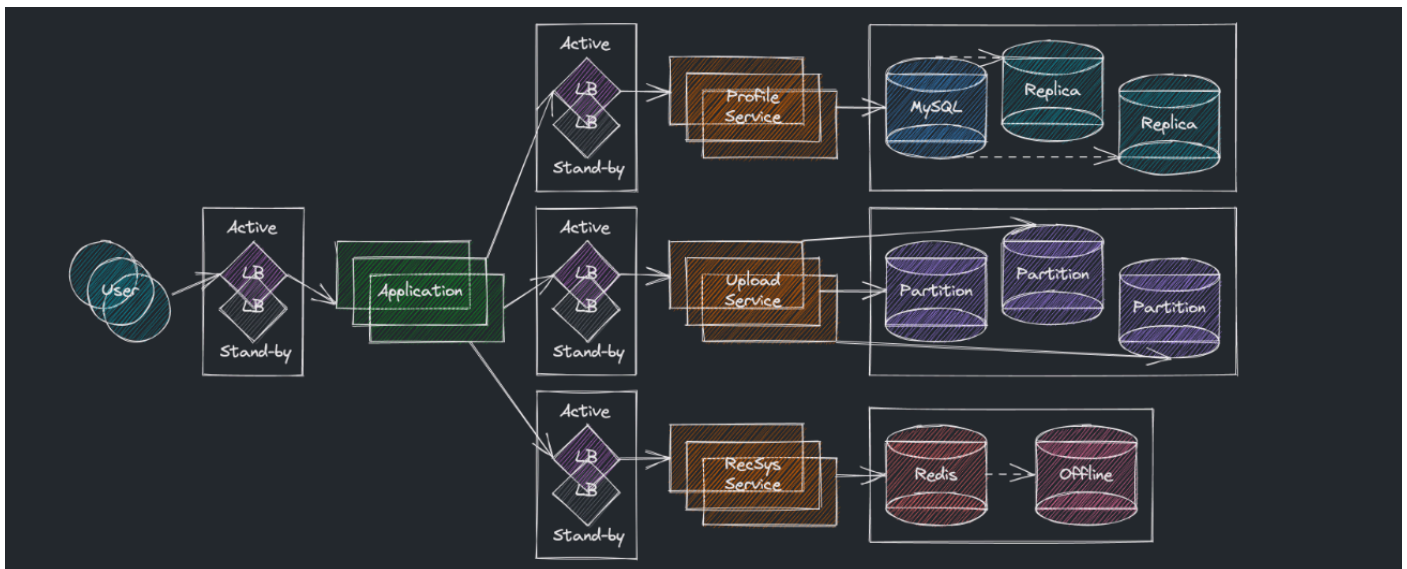
В случае консистентного хеширования ключей ранее рассмотренный пример будет выглядеть так, что ключи, которые были на 3х серверах и остались в рабочем состоянии, на них же и останутся. А ключи с 4-го сервера равномерно распределятся по 3-м оставшимся, мы потеряли ровно 25% ключей — возможный минимум.

> Сферический сервис в вакууме

Представим, что у нас есть простой сервис, который нужно подготовить к масштабированию. Из функционала есть редактирование профиля, заливка медиа и получение готовых рекомендаций.



Разберем, как можно масштабировать такой сервис:



- Приходит много пользователей, и их запросы проходят через loadbalancer. Также существует его реплика, которая находится в режиме ожидания и будет работать в случае поломки основного балансировщика;
- Балансировщик нагрузки выбирает, какому сервису передать входящий запрос;
- Также у приложения есть много разных инстансов, которые должны обрабатывать запросы за адекватное время. Если один из них перестаёт отвечать, то опционально мы передаём информацию в автоскейлер, который создаст новый инстанс;
- Дополнительные балансировщики нагрузки и их копии для сервисов. У каждого сервиса также есть инстансы;
- У баз данных есть реплики, которые помогут сервису продолжать работу в случае неполадок с оригинальной базой данных.

> Выводы

- При масштабировании и увеличении трафика появляется необходимость в балансировщике нагрузки
- При масштабировании и увеличении трафика появляется необходимость секционировать данные
- Для повышения надежности можно добавлять избыточные сервисы и реплицировать данные
- Есть разные парадигмы для синхронизации реплик, такие как кворум или лидер и последователи
- При шардировании нужно выбирать ключ так, чтобы был баланс и в нагрузке, и в занимаемом месте
- При генерации ключа для выбора реплики стоит прибегать к консистентному хешированию